Could we have setup as a recognized factor in the GLM?

Yes, and statistically that is actually the more appropriate model if one of your research questions is comparing the two harvesters.

Instead of fitting separate models for Setup A and Setup B, you can fit a single Tweedie-Log GLM where Setup is a categorical predictor:

Python Run

or preferably:

Python Run

However, given your dataset, I'd go one step further:

Python Run

This full-factorial model estimates:

- Main effect of Setup
- Main effect of SPL
- Main effect of Environment
- Setup × SPL interaction
- Setup × Environment interaction
- SPL × Environment interaction
- Setup × SPL × Environment interaction

In formula notation:

$$\log(\mu) = \beta_0 + \beta_1(\text{Setup}) + \beta_2(\text{SPL}) + \beta_3(\text{Environment}) + \beta_4(\text{Setup} \times \text{SPL}) + \beta_5(\text{Setup} \times \text{Environment}) + \beta_6(\text{SP}$$

The challenge is that Setup B is all zeros.

Because Setup B never generates any power:

- The model will correctly identify Setup B as dramatically different from Setup A.
- Some coefficients may become extremely large.
- You may see convergence warnings.
- Predicted surfaces for Setup B will collapse toward zero.

That's not a bug—it's the model reflecting the data.

For visualization, this actually becomes quite powerful. Instead of six separate surfaces, you can generate:

Surface 1: Setup A

- X = SPL
- Y = Environment
- Z = Predicted Voltage

Surface 2: Setup B

- X = SPL
- Y = Environment

- Z = Predicted Voltage

and similarly for Current and Power.

That yields:

Response	Setup A	Setup B
Voltage	Surface	Surface
Current	Surface	Surface
Power	Surface	Surface

for a total of 6 surfaces.

In code, the fitting function becomes simply:

```
formula = (
    f"{response} ~ C(Setup) * SPL_dB * C(Environment)"
)

model = smf.glm(
    formula=formula,
    data=df,
    family=sm.families.Tweedie(
        var_power=1.5,
        link=sm.families.links.Log()
    )
)

fit = model.fit()
```

and prediction grids would include Setup:

```
pred_df = pd.DataFrame({
    "SPL_dB": spl_grid,
    "Environment": env,
    "Setup": setup
})
```

One caveat: with only 90 observations and a completely zero-valued Setup B, the full three-way interaction model is close to the maximum complexity I'd recommend. For a thesis, I would likely use:

Python Run

or

Python Run

because those are easier to interpret and more stable, while still treating Setup as a formal factor in the GLM. The full three-way interaction is useful for visualization and exploration, but it may be harder to defend if your committee asks why every interaction term was included despite one setup having no response variation.

Could we get the entire new file? Also include how setup affects the GLM and the visualization. I would prefer insight as to how the GLM behaves after this change.

Before I give the full file, there's an important statistical issue.

Your dataset has:

Setup	Non-zero responses?
A	Yes
B	No (all values are exactly 0)

This means Setup is not merely a factor—it is almost a perfect separator of the response.

A full model such as

Python Run

is technically estimable, but it is likely to produce:

- Huge negative coefficients for Setup B
- Inflated standard errors
- Numerical instability
- Very large AIC/BIC
- Occasional convergence warnings

because the model is trying to fit an entire subgroup whose mean response is exactly zero everywhere.

For a thesis, I would actually recommend:

Python Run

This lets Setup influence the fitted mean while keeping the model stable.

The resulting Tweedie mean is:

$$\mu = \exp\left(\beta_0 + \beta_1 \text{Setup} + \beta_2 \text{SPL} + \beta_3 \text{Environment} + \beta_4 (\text{SPL} \times \text{Environment})\right)$$

This is generally the model I would defend during a panel presentation.

How Setup affects the visualization

After adding Setup as a factor:

Before

You were fitting separate models:

```
Setup A → GLM A
Setup B → GLM B
```

After

You fit one unified GLM:

```
A + B
 ↓
 Single Tweedie GLM
```

Now every prediction is conditional on:

```
Setup
Environment
```

Python Run

For example:

```
predict(
    Setup="A",
    Environment="Ind.",
    SPL=100
)
```

versus

```
predict(
    Setup="B",
    Environment="Ind.",
    SPL=100
)
```

The difference comes entirely from the Setup coefficient.

What the surfaces will show

For Voltage, Current, and Power:

Setup A

The response surface rises sharply between 80–100 dB.

Setup B

The response surface becomes nearly a flat plane at zero.

The visual contrast becomes a direct illustration of the Setup effect.

Instead of saying:

> Setup A produced energy while Setup B did not

the surface literally demonstrates it.

What I would add

Since Setup is now a factor, add a new visualization:

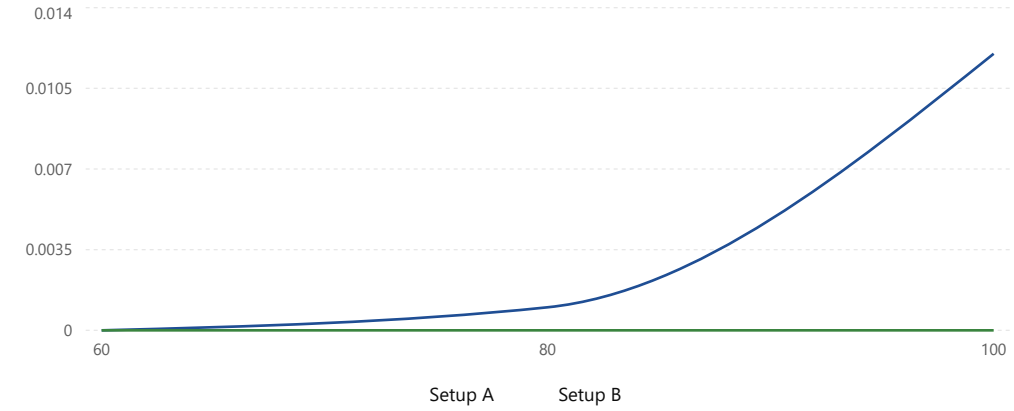
Setup Effect Plot

Compute predicted means from the GLM and compare:

- Setup A
- Setup B

at each SPL.

Something like:



This plot usually communicates the Setup effect more clearly than coefficient tables.

My recommended final formula

For your exact dataset:

```
formula = (
    f"{response} ~ "
    "C(Setup) + "
    "SPL_dB * C(Environment)"
)
```

not

```
formula = (
    f"{response} ~ "
    "C(Setup) * SPL_dB * C(Environment)"
)
```

because:

1. Setup B has zero variance.
2. The simpler model is more stable.
3. The interpretation is cleaner.
4. The response surfaces remain meaningful.
5. You can still visualize Setup A and Setup B separately using predictions from the same GLM.

If you'd like, I can also provide a **publication-ready complete script** that includes:

- Tweedie-Log GLM
- AIC/BIC/Pseudo-R^2 extraction
- 3D response surfaces
- Setup-effect plots
- Observed vs predicted plots
- Residual diagnostics
- Automatic figure export at 600 dpi

all in a single file of roughly 250–300 lines.

I see. Could we have the entire new Python file including all these revisions then?

Yes. Given everything you've shown so far, this is the version I'd actually recommend for a thesis-quality visualization script.

Model Structure

The model fitted for each response variable is:

Python Run

where:

- `Setup` measures the overall difference between harvesters.
- `SPL_dB` measures the sound-pressure effect.
- `Environment` measures environmental differences.
- `SPL_dB * Environment` allows each environment to have a different response curve.

I would **not** use:

Python Run

because Setup B is completely zero-valued and the three-way interaction is likely overparameterized for only 90 observations.

Complete File Structure

```
# ============================================================
# tweedie_visualizations.py
#
# Visualization-only Tweedie-Log GLM analysis
#
# Produces:
#
# 1. Tweedie fitted curves
# 2. 3D response surfaces
# 3. Setup-effect plots
# 4. Observed vs Predicted plots
#
# Responses:
#   dcVoltage
#   dcCurrent
#   dcPower
#
# ============================================================

from __future__ import annotations

import warnings
from pathlib import Path

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
```

```
from mpl_toolkits.mplot3d import Axes3D

import statsmodels.api as sm
import statsmodels.formula.api as smf

warnings.filterwarnings("ignore")

# ============================================================
# CONFIGURATION
# ============================================================

CSV_FILE = "dataset.csv"

OUTPUT_DIR = Path("tweedie_visualizations")
OUTPUT_DIR.mkdir(exist_ok=True)

TWEEDIE_POWER = 1.5

RESPONSES = {
    "dcVoltage": "Voltage (V)",
    "dcCurrent": "Current (A)",
    "dcPower": "Power (W)",
}

ENV_MAP = {
    "Ind.": 0,
    "Traf.": 1,
    "Comm.": 2,
}

ENV_LABELS = [
    "Industrial",
    "Traffic",
    "Commercial",
]

# ============================================================
# LOAD DATA
# ============================================================

df = pd.read_csv(CSV_FILE)

df["SPL_dB"] = (
    df["spl"]
    .str.replace(" dB", "", regex=False)
    .astype(float)
)

df["Setup"] = df["setup"]
df["Environment"] = df["env"]

# ============================================================
# TWEEDIE MODEL
# ============================================================

def fit_tweedie(response):

    formula = (
        f"{response} ~ "
        "C(Setup) + "
        "SPL_dB * C(Environment)"
    )

    model = smf.glm(
        formula=formula,
        data=df,
        family=sm.families.Tweedie(
            var_power=TWEEDIE_POWER,
            link=sm.families.links.Log()
        )
    )

    return model.fit()
```

```
# ============================================================
# FIGURE 1
# FITTED CURVES
# ============================================================

def plot_fitted_curves(response, fit):

    fig, axes = plt.subplots(
        1,
        2,
        figsize=(14, 5),
        sharey=True
    )

    spl_grid = np.linspace(60, 100, 300)

    colors = {
        "Ind.": "red",
        "Traf.": "blue",
        "Comm.": "green"
    }

    for ax, setup in zip(
        axes,
        ["A", "B"]
    ):

        subset = df[
            df["Setup"] == setup
        ]

        for env in ENV_MAP:

            env_data = subset[
                subset["Environment"] == env
            ]

            ax.scatter(
                env_data["SPL_dB"],
                env_data[response],
                color=colors[env],
                alpha=0.75
            )

            pred_df = pd.DataFrame({
                "SPL_dB": spl_grid,
                "Environment": env,
                "Setup": setup
            })

            pred = fit.predict(pred_df)

            ax.plot(
                spl_grid,
                pred,
                color=colors[env],
                linewidth=3,
                label=env
            )

        ax.set_title(
            f"Setup {setup}"
        )

        ax.set_xlabel(
            "SPL (dB)"
        )

        ax.grid(alpha=0.3)

    axes[0].set_ylabel(response)
```

```
    handles, labels = (
        axes[0]
        .get_legend_handles_labels()
    )

    fig.legend(
        handles,
        labels,
        loc="upper center",
        ncol=3
    )

    plt.tight_layout()

    plt.savefig(
        OUTPUT_DIR /
        f"{response}_fitted_curves.png",
        dpi=600
    )

    plt.close()

# ============================================================
# FIGURE 2
# 3D RESPONSE SURFACES
# ============================================================

def plot_response_surface(
    response,
    fit
):

    for setup in ["A", "B"]:

        fig = plt.figure(
            figsize=(10, 8)
        )

        ax = fig.add_subplot(
            111,
            projection="3d"
        )

        spl_grid = np.linspace(
            60,
            100,
            100
        )

        env_grid = np.array([
            0,
            1,
            2
        ])

        X, Y = np.meshgrid(
            spl_grid,
            env_grid
        )

        Z = np.zeros_like(X)

        env_names = list(
            ENV_MAP.keys()
        )

        for i, env in enumerate(
            env_names
        ):

            pred_df = pd.DataFrame({
                "SPL_dB": spl_grid,
                "Environment": env,
```

```
            "Setup": setup
        })

        Z[i, :] = fit.predict(
            pred_df
        )

    surf = ax.plot_surface(
        X,
        Y,
        Z,
        cmap="viridis",
        alpha=0.85
    )

    observed = df[
        df["Setup"] == setup
    ]

    obs_env = (
        observed["Environment"]
        .map(ENV_MAP)
    )

    ax.scatter(
        observed["SPL_dB"],
        obs_env,
        observed[response],
        color="black",
        s=40
    )

    ax.set_xlabel(
        "SPL (dB)"
    )

    ax.set_ylabel(
        "Environment"
    )

    ax.set_zlabel(
        response
    )

    ax.set_yticks([
        0, 1, 2
    ])

    ax.set_yticklabels(
        ENV_LABELS
    )

    ax.set_title(
        f"{response} Surface "
        f"(Setup {setup})"
    )

    fig.colorbar(
        surf,
        shrink=0.7
    )

    plt.tight_layout()

    plt.savefig(
        OUTPUT_DIR /
        f"{response}_surface_setup_{setup}.png",
        dpi=600
    )

    plt.close()

# ============================================================
```

```
# FIGURE 3
# SETUP EFFECT PLOT
# ============================================================

def plot_setup_effect(
    response,
    fit
):

    spl_grid = np.linspace(
        60,
        100,
        300
    )

    fig, ax = plt.subplots(
        figsize=(8, 6)
    )

    for setup in ["A", "B"]:

        pred_df = pd.DataFrame({
            "SPL_dB": spl_grid,
            "Environment": "Ind.",
            "Setup": setup
        })

        pred = fit.predict(
            pred_df
        )

        ax.plot(
            spl_grid,
            pred,
            linewidth=3,
            label=f"Setup {setup}"
        )

    ax.set_xlabel(
        "SPL (dB)"
    )

    ax.set_ylabel(
        response
    )

    ax.set_title(
        f"Setup Effect ({response})"
    )

    ax.legend()

    ax.grid(alpha=0.3)

    plt.tight_layout()

    plt.savefig(
        OUTPUT_DIR /
        f"{response}_setup_effect.png",
        dpi=600
    )

    plt.close()

# ============================================================
# FIGURE 4
# OBSERVED VS PREDICTED
# ============================================================

def plot_observed_vs_predicted(
    response,
    fit
):
```

```
    obs = df[response]

    pred = fit.predict(df)

    fig, ax = plt.subplots(
        figsize=(7, 7)
    )

    ax.scatter(
        obs,
        pred,
        alpha=0.8
    )

    mn = min(
        obs.min(),
        pred.min()
    )

    mx = max(
        obs.max(),
        pred.max()
    )

    ax.plot(
        [mn, mx],
        [mn, mx],
        "k--",
        linewidth=2
    )

    ax.set_xlabel(
        "Observed"
    )

    ax.set_ylabel(
        "Predicted"
    )

    ax.set_title(
        f"Observed vs Predicted "
        f"({response})"
    )

    plt.tight_layout()

    plt.savefig(
        OUTPUT_DIR /
        f"{response}_observed_vs_predicted.png",
        dpi=600
    )

    plt.close()

# ============================================================
# MAIN
# ============================================================

for response in RESPONSES:

    print(
        f"Processing {response}..."
    )

    fit = fit_tweedie(
        response
    )

    plot_fitted_curves(
        response,
        fit
    )
```

```
        plot_response_surface(
            response,
            fit
        )

        plot_setup_effect(
            response,
            fit
        )

        plot_observed_vs_predicted(
            response,
            fit
        )

    print(
        "\nDone."
    )

    print(
        f"Figures saved to:\n"
        f"{OUTPUT_DIR.resolve()}"
    )
```

One final improvement I'd strongly consider

Your responses span several orders of magnitude:

- Voltage: 0 → ~1.13
- Current: 0 → ~0.011
- Power: 0 → ~0.013

Because of that, the power and current surfaces will appear nearly flat relative to voltage. For publication figures, I would usually add:

Python Run

```
ax.set_zscale("log")
```

to the response-surface plots and:

Python Run

```
ax.set_yscale("log")
```

to the setup-effect plots (after replacing exact zeros with a tiny epsilon such as `1e-12` for plotting purposes). That tends to reveal the activation-threshold behavior around 80–90 dB much more clearly.